



Special Course in Software Engineering '26

Project – Requirements Engineering

Important

- Work in groups
 - Therefore, you must sit with your group members
 - Communicating over WeChat or similar application is not acceptable
 - If you don't like this, then you are free to leave, and you'll be marked absent

Convert human brief into requirements

- Download the **brief.txt** document
- This document has human-language requirements that you must build the software for using your agents
- First, ask **Qwen** to read your text and have it produce software requirements from the text. This is similar to the lost-and-found exercise.
 - Run the program several times and examine the output. Is it consistent?
 - Save the output in a text file called **robot_requirements.txt**
 - Save the Python file as **brief_to_req.py**
- Is the above output acceptable?
 - Can that output be used directly by another program?

Analyst Agent

- This will be your first (almost) agent. Save it as **analyst_agent.py**
- The only responsibility of this agent is to analyze the brief text and convert it into explicit software requirements
 - This is like a human requirements engineer. Their job is to take the customer's requirements text, and extract specific requirements from it
- The rest of your system must use output from Analyst Agent. Therefore, it must produce valid **JSON** using exactly the following structure:

```
{  
  "goal": "string",  
  "allowed_actions": ["FORWARD", "LEFT", "RIGHT", "STOP"],  
  "safe_stop": true,  
  "avoid_obstacles": true  
}
```

- The **"goal"** stores the navigation objective (as a string) understood from the brief text, **"allowed_actions"** are the valid steps the robot can take, **"safe_stop"** means if the robot should stop safely when it cannot go anywhere, and **"avoid_obstacles"** states if the robot must avoid anything that blocks its path.

Analyst Agent (CTD.)

- Besides the the valid JSON the Agent must produce, the final navigation logic system from the system must always be only: **FORWARD**, **LEFT**, **RIGHT**, **STOP**
- There should be no additional action item

Improve Analyst Agent

- Based on the description and the conditions given to you so far, create a **suitable prompt for it**.
- The prompt should ideally contain
 - Role of the model
 - What it must do
 - The constraints and rules it must follow
 - The final output structure
- Remember, the above final output **MUST** come from Qwen and not your Python program.

Improve Analyst Agent (CTD.)

- Create a function called `run_analyst()`. The function must
 - Accept the brief text as parameter. Therefore, it's called by some other function
 - Send appropriate prompts to the Qwen model
 - Accept Qwen's response
 - Convert JSON text into Python data (HINT: Use `json.loads()`)
 - Validate the data
 - Return validated requirements

Improve Analyst Agent (CTD.)

- For validation of Qwen output, create another function called `validate_requirements()`
 - The function takes the result, which you converted into a Python data structure, as a parameter
 - The function must check that the above data is a valid dictionary
 - The function must check that all the necessary keys (`goal`, `allowed_actions`, etc.) exist. It must also ensure the following: (HINT: Use `isinstance()`)
 - Goal is a `string`
 - Allowed_actions is a `list`
 - Both `safe_stop` and `avoid_obstacles` is a `boolean`
 - The function must ensure there are no missing keys or some new key was introduced by Qwen
- Overall, the program must make sure that Qwen produced a `valid JSON` and next that this JSON satisfies the software requirements

Runner

- Create a small Python file called `run_analyst.py`. Its responsibility is to
 - Read the brief text
 - Call `run_analyst()`
 - Receive final validated software requirements
 - Save the requirements as a `requirements.json` file inside a folder called `artifacts`. (HINT: use `json.dump()`)

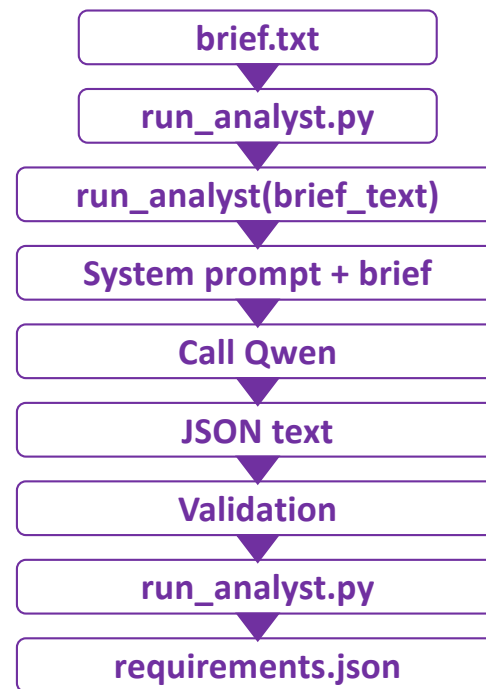
Overall Structure

- Overall, your file structure should look like the following:

```
▼ agentic_se
  ▼ artifacts
    | {} requirements.json
    |
    | 🐍 analyst_agent.py
    | ≡ brief-txt
    |
    | 🐍 run_analyst.py
```

The complete flow

- Overall, the execution flow of the program would be as follows:



Submission

- Create a GitHub repository with the name `scse_agentic_se_groupname`
 - Replace `groupname` with your actual group name.
- All the files mentioned in this instruction must be available in the repository
- Submit only the GitHub repository link to Moodle by **September 23, Midnight**

That's all for this practice